

UNIVERSIDAD AUTONOMA DE SINALOA



Licenciatura en Informática

Materia: Sistemas Distribuidos

Prof. Jesus Humberto Abundis Patiño

Alumno:

Galindo Melendrez Raul Caleb

Galindo Melendrez Sergio Jesus

Higuera Medina Jocelyn Guadalupe

Grado y grupo: 5-2

Tema: Proyecto Final 100 libros

Fecha: 08/12/2025



Link de archivos: https://github.com/SergioUAS/Proyecto_Final_100_Libros.git

Contenido

1. INTRODUCCIÓN Y ENTORNO DE DESARROLLO	3
1.1 Descripción del Proyecto.....	3
1.2 Configuración del Entorno	3
2. FASE 1: OBTENCIÓN AUTOMATIZADA DE DATOS (ETL)	4
2.1 Web Scraping.....	4
3. FASE 2: PRE-PROCESAMIENTO Y LIMPIEZA DE DATOS (ETL)	5
3.1 Eliminación de Metadatos (Script <code>clean_books.py</code>).....	5
3.2 Normalización y Creación de Vocabulario (Script <code>create_vocab.py</code>)	5
4. FASE 3: PROCESAMIENTO DISTRIBUIDO CON PYSPARK	6
4.1 Vectorización (CountVectorizer).....	6
4.2 Cálculo de Similitud (Matriz de Similitud)	6
4.3 Justificación Técnica: ¿Por qué PySpark?	7
5. FASE 4: RESULTADOS Y APLICACIÓN.....	7
5.1 Sistema de Recomendación (<code>recommend.py</code>)	7
5.2 Resumen Automático por Palabras Clave (<code>summarize_book.py</code>).....	8
6. CONCLUSIONES	10

1. INTRODUCCIÓN Y ENTORNO DE DESARROLLO

1.1 Descripción del Proyecto

El objetivo de este proyecto es desarrollar un sistema capaz de procesar lenguaje natural (NLP) utilizando técnicas de Big Data. El sistema automatiza la descarga de literatura clásica, limpia los textos, calcula similitudes matemáticas entre libros para generar recomendaciones y crea resúmenes automáticos basados en la relevancia de palabras clave (TF-IDF).

1.2 Configuración del Entorno

Para garantizar la reproducibilidad y el manejo eficiente de datos, el proyecto se desarrolló sobre un subsistema Linux (Ubuntu en WSL). Se utilizó un entorno virtual de Python para aislar las dependencias y Java (JDK) como requisito fundamental para la ejecución del motor de procesamiento distribuido Apache Spark.

- **Lenguaje:** Python 3.x
- **Motor de Datos:** PySpark
- **Gestión de Entorno:** venv (Virtual Environment)
- **Sistema Operativo:** Ubuntu (WSL)

```
* sergiogalindo@SergioGalindo:~/proyecto_100_libros$ source /home/sergiogalindo/proyecto_100_libros/venv/bin/activate
* (venv) sergiogalindo@SergioGalindo:~/proyecto_100_libros$ java -version
openjdk version "21.0.9" 2025-10-21
OpenJDK Runtime Environment (build 21.0.9+10-Ubuntu-124.04)
OpenJDK 64-Bit Server VM (build 21.0.9+10-Ubuntu-124.04, mixed mode, sharing)
* (venv) sergiogalindo@SergioGalindo:~/proyecto_100_libros$ █
```

2. FASE 1: OBTENCIÓN AUTOMATIZADA DE DATOS (ETL)

2.1 Web Scraping

En lugar de descargar los libros manualmente, se implementó un script de extracción automática (`get_books.py`). Este script utiliza las librerías `requests` y `BeautifulSoup` para conectarse a los servidores de Project Gutenberg, identificar los enlaces de los libros más populares ("Top 100") y descargarlos secuencialmente.

El proceso asegura que siempre se trabaje con la información más actual disponible en la fuente y normaliza los nombres de los archivos para facilitar su procesamiento posterior.

```
(venv) sergiogalindo@SergioGalindo:~/proyecto_100_libros$ python3 get_books.py
Descargado: Bleak_House_by_Charles_Dickens.txt
Descargado: The_Souls_of_Black_Folk_by_W_E_B_Du_Bois.txt
Descargado: The_Wars_of_Religion_in_France_15591576_by_James_Westfall_Thompson.txt
Descargado: Puvis_de_Chavannes_by_François_Craistre.txt
Descargado: Treasure_Island_by_Robert_Louis_Stevenson.txt
Descargado: Anna_Karenina_by_graf_Leo_Tolstoy.txt
Descargado: On_Liberty_by_John_Stuart_Mill.txt
Descargado: The_country_seats_of_the_United_States_of_North_America_by_William_Birch.txt
Descargado: The_divine_comedy_by_Dante_Alighieri.txt
Descargado: The_Romance_of_Lust_A_classic_Victorian_erotic_novel_by_Anonymous.txt
Descargado: Gullivers_Travels_into_Several_Remote_Nations_of_the_World_by_Jonathan_Swift.txt
Descargado: The_Republic_by_Plato.txt
Descargado: The_Yellow_Wallpaper_by_Charlotte_Perkins_Gilman.txt
Descargado: Society_in_America_Volume_1_of_2_by_Harriet_Martineau.txt
Descargado: The_Part_Borne_by_the_Dutch_in_the_Discovery_of_Australia_16061765_by_J_E_Heeres.txt
Descargado: The_Odyssey_by_Homer.txt
Descargado: The_Confessions_of_St_Augustine_by_Bishop_of_Hippo_Saint_Augustine.txt
Descargado: Leibnizs_New_Essays_Concerning_the_Human_Understanding_A_Critical_Exposition_by_John_Dewey.txt
Descargado: A_Study_in_Scarlet_by_Arthur_Conan_Doyle.txt
Descargado: Grimms_Fairy_Tales_by_Jacob_Grimm_and_Wilhelm_Grimm.txt
Descargado: Paradise_Lost_by_John_Milton.txt
Descargado: Romantic_castles_and_palaces.txt
Descargado: Meditations_by_Emperor_of_Rome_Marcus_Aurelius.txt
Descargado: The_Kama_Sutra_of_Vatsyayana_by_Vatsyayana.txt
Descargado: Don_Quixote_by_Miguel_de_Cervantes_Saavedra.txt
Descargado: Blue_trousers_by_Murasaki_Shikibu.txt
Descargado: Oliver_Twist_by_Charles_Dickens.txt
Descargado: Sense_and_Sensibility_by_Jane_Austen.txt
Descargado: Little_Women_by_Louisa_May_Alcott.txt
Descargado: Shakespeares_family_by_C_C_Stopes.txt
Descargado: The_Philosophy_of_Auguste_Comte_by_Lucien_LévyBruhl.txt
Descargado: 冷眼观_by_Junqing_Wang.txt
Descargado: Doctrina_Christiana.txt
Descargado: Tess_of_the_durbervilles_A_Pure_Woman_by_Thomas_Hardy.txt
Descargado: Les_Misérables_by_Victor_Hugo.txt
Descargado: Harriet_Martineau_by_Florence_Ferwick_Miller.txt
Descargado: The_Tragical_History_of_Doctor_Faustus_by_Christopher_Marlowe.txt
Descargado: War_and_Peace_by_graf_Leo_Tolstoy.txt
¡listo!
(venv) sergiogalindo@SergioGalindo:~/proyecto_100_libros$
```

3. FASE 2: PRE-PROCESAMIENTO Y LIMPIEZA DE DATOS (ETL)

Una vez descargados los textos brutos, se aplicó un proceso de limpieza en dos etapas para asegurar la calidad de la información antes de procesarla matemáticamente.

3.1 Eliminación de Metadatos (Script `clean_books.py`)

Los archivos de Project Gutenberg incluyen licencias, introducciones legales y pies de página que no forman parte de la obra literaria y agregarían "ruido" al análisis.

Para solucionar esto, se implementó un algoritmo que:

1. Lee cada archivo de texto descargado.
2. Localiza los marcadores estandarizados de inicio (ej. `*** START OF...`) y fin (ej. `*** END OF...`).
3. Recorta el texto para conservar únicamente el contenido narrativo entre esos dos puntos.

```
(venv) sergiogalindo@SergioGalindo:~/proyecto_100_libros$ python3 clean_books.py
Limpiando 100 libros...
Limpieza terminada.
(venv) sergiogalindo@SergioGalindo:~/proyecto_100_libros$
```

3.2 Normalización y Creación de Vocabulario (Script `create_vocab.py`)

Para que el modelo matemático pueda comparar los libros, fue necesario estandarizar el lenguaje. Se aplicaron cuatro filtros fundamentales de Procesamiento de Lenguaje Natural (NLP):

1. **Minúsculas:** Se transformó todo el texto a caja baja (lowercase) para evitar que "Hola" y "hola" se cuenten como palabras diferentes.
2. **Limpieza de Caracteres:** Mediante Expresiones Regulares (Regex), se eliminaron signos de puntuación, números y símbolos especiales, dejando solo caracteres alfabéticos.
3. **Tokenización:** Se dividió el texto continuo en unidades individuales (tokens/palabras).
4. **Eliminación de Stop Words:** Se filtraron palabras funcionales muy comunes en inglés (como "the", "and", "is", "at") que no aportan significado semántico relevante para distinguir un libro de otro.

```
(venv) sergiogalindo@SergioGalindo:~/proyecto_100_libros$ python3 create_vocab.py
Procesando vocabulario de 100 libros...
Vocabulario creado con 198995 palabras unicas.
Guardado en 'vocabulario.txt'
(venv) sergiogalindo@SergioGalindo:~/proyecto_100_libros$
```

4. FASE 3: PROCESAMIENTO DISTRIBUIDO CON PYSPARK

Esta es la fase central del proyecto, donde se transforman los datos textuales limpios en estructuras matemáticas procesables para encontrar patrones.

4.1 Vectorización (CountVectorizer)

Para que una computadora pueda comparar libros, primero debe convertir el texto en números. Utilizando la librería `pyspark.ml.feature`, aplicamos un proceso de **Vectorización**.

Si nuestro vocabulario consta de N palabras únicas, cada libro se representa como un vector de dimensión N . En este vector, cada posición corresponde a una palabra específica y el valor numérico indica la frecuencia de dicha palabra en el libro. Esto convierte cada obra literaria en una coordenada en un espacio multidimensional.

4.2 Cálculo de Similitud (Matriz de Similitud)

Una vez que los libros son vectores numéricos, utilizamos la **Similitud del Coseno** para medir la cercanía entre ellos.

- Esta métrica calcula el coseno del ángulo entre dos vectores.
- Un valor de **1.0** indica que los vectores (libros) son idénticos en composición.
- Un valor de **0.0** indica que no comparten vocabulario relevante.

El resultado final de este proceso es una matriz cuadrada (100×100) que almacena el grado de similitud de cada libro contra todos los demás.

4.3 Justificación Técnica: ¿Por qué PySpark?

Aunque este prototipo procesa 100 libros, la arquitectura fue diseñada para ser escalable. Al utilizar **Apache Spark** y sus RDDs (Resilient Distributed Datasets), este mismo código puede ejecutarse en un clúster de servidores para procesar bibliotecas masivas (como los más de 60,000 libros de Gutenberg) dividiendo la carga de trabajo, algo que sería ineficiente o imposible por falta de memoria RAM con scripts secuenciales tradicionales de Python.

```
(venv) sergiogalindo@sergiogalindo:~/proyecto_100_libros$ python3 matrix_spark.py
/home/sergiogalindo/proyecto_100_libros/matrix_spark.py:35: SyntaxWarning: invalid escape sequence '\V'
  df = df.withColumn("title", regexp_replace("path", ".*\V", ""))
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
25/12/08 02:55:12 WARN Utils: Your hostname, Sergiogalindo, resolves to a loopback address: 127.0.1.1; using 10.255.255.254 instead (on interface lo)
25/12/08 02:55:12 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
25/12/08 02:55:12 WARN NativeCodeLoader: unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Calculando matriz 100x100...
Matriz calculada y guardada en 'matriz_similitud.txt'
(venv) sergiogalindo@sergiogalindo:~/proyecto_100_libros$
```

5. FASE 4: RESULTADOS Y APLICACIÓN

Finalmente, se desarrollaron dos herramientas de consulta que explotan la información procesada para generar valor al usuario final.

5.1 Sistema de Recomendación (`recommend.py`)

Este módulo carga la matriz de similitud pre-calculada y permite consultar cualquier título de la base de datos. El algoritmo identifica los vectores con menor distancia angular (mayor similitud coseno) respecto al libro consultado y devuelve los 5 títulos más afines.

Las pruebas realizadas demuestran la coherencia temática del sistema. Por ejemplo, al buscar *A Christmas Carol* de Charles Dickens, el sistema recomendó correctamente otras obras del mismo autor y novelas de la época victoriana, validando la eficacia de la vectorización.

```
(venv) sergiogalindo@SergioGalindo:~/proyecto_100_libros$ python3 recommend.py
--- Recomendando libros basados en: A_Christmas_Carol_in_Prose_Being_a_Ghost_Story_of_Christmas_by_Charles_Dickens.txt ---
1. A Tale of Two Cities by Charles Dickens.txt
2. A Study in Scarlet by Arthur Conan Doyle.txt
3. Tess of the d'Urbervilles A Pure Woman by Thomas Hardy.txt
4. Oliver Twist by Charles Dickens.txt
5. Ulysses by James Joyce.txt
(venv) sergiogalindo@SergioGalindo:~/proyecto_100_libros$
```


5.2 Resumen Automático por Palabras Clave (`summarize_book.py`)

Para sintetizar el contenido de los libros, se implementó un algoritmo **TF-IDF** (Term Frequency - Inverse Document Frequency). A diferencia de un conteo simple de palabras, este método penaliza los términos que aparecen en todos los libros y premia aquellos que son específicos de la obra analizada.

El resultado es una lista de palabras clave que describen la trama. En las pruebas, el resumen de *A Christmas Carol* arrojó términos como "Scrooge", "Ghost" y "Cratchit", capturando la esencia de la historia sin intervención humana.

```
(venv) sergiogalindo@SergioGalindo:~/proyecto_100_libros$ python3 summarize_book.py "Christmas" 10
/home/sergiogalindo/proyecto_100_libros/summarize_book.py:33: SyntaxWarning: invalid escape sequence '\V'
  df = df.withColumn("title", regexp_replace("path", ".*\V", ""))
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
25/12/08 02:58:39 WARN Utils: Your hostname, SergioGalindo, resolves to a loopback address: 127.0.0.1; using 10.255.255.254 instead (on interface lo)
25/12/08 02:58:39 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
25/12/08 02:58:40 WARN NativeCodeLoader: unable to load native-hadoop library for your platform... using builtin-java classes where applicable

-- Palabras resumen para: A_Christmas_Carol_in_Prose_Being_a_Ghost_Story_of_Christmas_by_Charles_Dickens.txt --
1. scrooge
2. scrooges
3. cratchit
4. bob
5. christmas
6. tim
7. ghost
8. joe
9. nephew
10. jacob
(venv) sergiogalindo@SergioGalindo:~/proyecto_100_libros$
```

Análisis del Resultado:

1. **Identificación Correcta:** El sistema buscó en la base de datos y encontró automáticamente el libro *A Christmas Carol in Prose* de Charles Dickens, demostrando que la búsqueda por coincidencia de texto funciona.
2. **Extracción de Palabras Clave (Keywords):** La lista numerada del 1 al 10 no muestra palabras comunes (como "el", "la", "casa"), sino los términos que definen la identidad de esta obra específica:
 - **Personajes:** *Scrooge* (protagonista), *Cratchit* (empleado), *Bob* (Bob Cratchit), *Tim* (Tiny Tim), *Jacob* (Jacob Marley), *Nephew* (el sobrino Fred).
 - **Temas:** *Ghost* (los fantasmas son el motor de la trama), *Christmas* (el tema central).


```

(venv) sergiogalindo@sergiogalindo:~/proyecto_100_libros$ python3 summarize_book.py "Blue" 10
/home/sergiogalindo/proyecto_100_libros/summarize_book.py:33: SyntaxWarning: invalid escape sequence '\ '
  df = df.withColumn("title", regexp_replace("path", ".*\\", ""))
WARNING: using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
25/12/08 03:01:33 WARN Utils: Your hostname, sergiogalindo, resolves to a loopback address: 127.0.0.1; using 10.255.255.254 instead (on interface lo)
25/12/08 03:01:33 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
25/12/08 03:01:33 WARN NativeCodeLoader: Unable to load native-heap library for your platform... using builtin-java classes where applicable

--- Palabras resumen para: Blue_trousers_by_Murasaki_Shikibu.txt ---
1. genji
2. ygirl
3. kashiwagi
4. chj
5. murasaki
6. nyosan
7. genjis
8. suraku
9. ochiba
10. kumoi
(venv) sergiogalindo@sergiogalindo:~/proyecto_100_libros$

```

Análisis del Resultado:

1. **Búsqueda y Coincidencia:** El sistema escaneó la carpeta de descargas y encontró el archivo `Blue_trousers_by_Murasaki_Shikibu.txt`. Esto confirma que la función de búsqueda de títulos mediante coincidencia de texto (substring matching) funciona correctamente.
2. **Generación del Resumen (TF-IDF):** Al procesar este texto, el algoritmo identificó las palabras clave de la obra *The Tale of Genji* (La Historia de Genji), escrita por Murasaki Shikibu.
 - Las palabras resultantes como **"genji"**, **"murasaki"**, **"kashiwagi"** y **"nyosan"** son los nombres de los personajes principales de esta literatura clásica japonesa.

6. CONCLUSIONES

El desarrollo de este proyecto permitió integrar el ciclo completo de la Ciencia de Datos: desde la extracción automatizada (ETL) hasta la aplicación de algoritmos de Machine Learning en textos (NLP).

Si bien se trabajó con una muestra de 100 libros, la decisión arquitectónica de utilizar **Apache Spark** es fundamental. Al procesar los datos mediante RDDs y operaciones distribuidas, este mismo código es capaz de escalar horizontalmente. Esto significa que si quisiéramos procesar la biblioteca completa de Project Gutenberg (más de 60,000 libros), no sería necesario reescribir la lógica del programa, sino simplemente ejecutarlo en un clúster con mayor capacidad de cómputo.

Este proyecto demuestra cómo las técnicas de Big Data permiten transformar datos no estructurados (texto plano) en información estructurada y útil (recomendaciones y resúmenes) de manera eficiente y escalable.